

## Question 1

### Steps

Heuristic: Counts the number of positions where the current word differs from the goal word.

Admissible because each incorrect position requires at least one move.

#### 1. Pseudocode

Differences = 0

For each position in current word:

    If the letter differs from the goal:

        +1 to differences

Return differences

#### 2. Unavoidable remaining cost

Each action changes only one letter and has a cost of 1. Every incorrect position must be changed at least once to reach the goal. Thus the number of incorrect positions gives a lower bound on the remaining cost.

#### 3. Why it can't overestimate

The heuristic is admissible because it can't overestimate the true minimum remaining cost. Each wrong letter needs at least one move. Extra moves may be needed to keep the words valid. The estimate can be lower than the actual cost but never higher.

### Scrabble

Heuristic: Adds the Scrabble values of the required goal letters at every incorrect position.

Admissible because every incorrect position must eventually be replaced with its goal letter, paying at least that letter's cost.

#### 1. Pseudocode

Set total = 0

For each position in current word:

    If that position/letter differs from the goal:

        Find the Scrabble cost of the goal letter

        Add that cost to total

Return total

### 2. Unavoidable remaining cost

Every incorrect position eventually is changed to its goal letter. Each replacement has a Scrabble cost based on the newly inserted letter. Adding these letter costs gives a lower bound on the remaining cost.

### 3. Why it can't overestimate

The heuristic is admissible because every incorrect position must eventually receive its goal letter, so those costs cannot be avoided. Additional moves may be needed to form valid dictionary words, increasing the actual cost. The estimate can be lower than the actual cost, but never higher.

## Frequency

Heuristic: Counts the number of incorrect positions and adds the rarity of the goal word.

Admissible because every incorrect position requires at least one move, and entering the goal word has an unavoidable rarity cost.

### 1. Pseudocode

Set differences = 0

If the current word is the goal:

Return 0

For each position in the current word:

If the letter differs from the goal:

+1 to differences

Return differences + rarity of the goal word

### 2. Unavoidable remaining cost

Each move costs 1 plus the rarity of the resulting word. Every incorrect position requires at least one move and reaching the goal requires paying the goal word's rarity. The minimum number of necessary moves + the goal's rarity gives a lower bound on the remaining cost.

### 3. Why it can't overestimate

The heuristic is admissible because every incorrect position requires at least one move and the goal's rarity cost cannot be avoided. Thus, the heuristic can underestimate the actual remaining cost, but cannot overestimate it. When the current word is already the goal, the heuristic returns 0 because no more moves are necessary.

## Question 2

| Word pair   | Cost measure | UCS cost | UCS nodes expanded | A* cost | A* nodes expanded | Reduction (%) |
|-------------|--------------|----------|--------------------|---------|-------------------|---------------|
| WHY → ASK   | Steps        | 5        | 773                | 5       | 16                | 97.93         |
| WHY → ASK   | Scrabble     | 9        | 765                | 9       | 13                | 98.30         |
| WHY → ASK   | Frequency    | 38.86    | 808                | 38.86   | 634               | 21.53         |
| BEAR → DUCK | Steps        | 4        | 691                | 4       | 6                 | 99.13         |
| BEAR → DUCK | Scrabble     | 11       | 2981               | 11      | 6                 | 99.80         |
| BEAR → DUCK | Frequency    | 38.24    | 1084               | 38.24   | 302               | 72.14         |

1. Yes, both returned the same optimal cost for all six tests. Different paths are acceptable as long as they have the same minimum total cost.
2. Scrabble for both word pairs.  
WHY → ASK: 98.30% reduction.  
BEAR → DUCK: 99.80% reduction.
3. Frequency for both word pairs.  
WHY → ASK: 21.53% reduction.  
BEAR → DUCK: 72.14% reduction.
4.  
Steps: Counts incorrect letter positions. Gives A\* a minimum number of remaining moves. Strong guidance when few extra moves are needed.  
Scrabble: Adds the cost of required goal letters. Accounts for expensive replacements. A\* expanded fewer nodes for both word pairs.  
Frequency: Counts incorrect positions plus the goal's rarity. Ignores rarity costs of intermediate words. Often underestimates the remaining cost by more, giving A\* less guidance.

## Question 3

### Steps

1. Yes.
2. Each action changes one letter and costs 1, regardless of direction. Reversing a path requires the same number of moves. Therefore, the optimal costs must be equal in both directions.

3. Counterexample:  
None. Steps costs are always symmetric.

## Scrabble

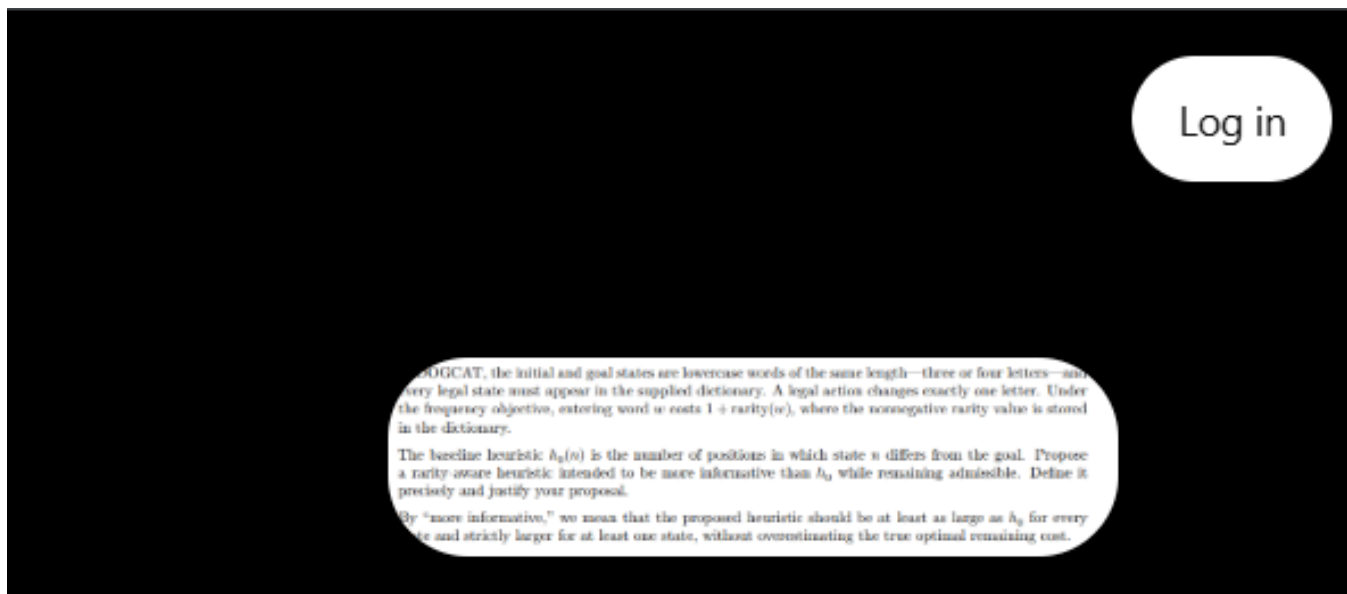
1. No.
2. The cost depends on the newly inserted letter. Reversing an action replaces a different letter, which may have a different Scrabble value. Therefore, optimal costs are not always equal.
3. Counterexample:  
dog  $\rightarrow$  cog: replacing d with c costs 3.  
cog  $\rightarrow$  dog: replacing c with d costs 2.

## Frequency

1. No.
2. Each action costs 1 plus the rarity of the resulting word. Reversing a path changes which word's rarity is paid upon arrival. Different starting and goal words may have different rarity values. Therefore, optimal costs are not always equal.
3. Counterexample:  
dog  $\rightarrow$  cog:  $1 + 11.39 = 12.39$   
cog  $\rightarrow$  dog:  $1 + 5.82 = 6.82$

## Question 4

1.  
Model: ChatGPT, free version. The exact model name was not recorded.  
Prompt: A screenshot of the exact prompt



Proposed heuristic: Count the incorrect letter positions. Multiply by 1 plus the smallest rarity value in the dictionary.

$$h(n) = (1 + r_{\min}) d(n, g).$$

2.

Find the smallest rarity in the dictionary  
Set differences = 0

For each position in the current word:

If the letter differs from the goal:

+1 to differences

Return differences \* (1 + minimum rarity)

3. The heuristic is admissible for the frequency cost system. Each action changes one letter. At least one move required for every incorrect position. Each move costs 1 plus the rarity of the resulting word. Every word has a rarity at least as large as the minimum. Therefore, the heuristic cannot overestimate the actual remaining cost. Additional moves and larger rarity values only increase the true cost. Since the minimum rarity is positive, the proposed heuristic is strictly larger than the baseline for every non-goal state.
4. Valid for this dictionary. The minimum rarity is positive. The original AI explanation about one positive word was incorrect, but the heuristic itself works. No replacement needed.

| Word pair   | Heuristic | Solution cost | Nodes expanded |
|-------------|-----------|---------------|----------------|
| WHY → ASK   | Baseline  | 38.86         | 755            |
| WHY → ASK   | Proposed  | 38.86         | 754            |
| BEAR → DUCK | Baseline  | 38.24         | 720            |
| BEAR → DUCK | Proposed  | 38.24         | 714            |

Both heuristics returned the same optimal costs. The proposed heuristic expanded 1 fewer node for WHY → ASK and 6 fewer for BEAR → DUCK. Small improvement because the dictionary's minimum rarity is very low. Unlike the frequency heuristic in Question 1, the proposed heuristic uses the minimum dictionary rarity instead of the goal word's rarity.

## Collaboration, sources, and generative AI policy

Sources used:

<https://www.youtube.com/watch?v=ySN5Wnu88nE>

[geeksforgeeks.org/python/string-slicing-in-python/](https://www.geeksforgeeks.org/python/string-slicing-in-python/)  
<https://www.geeksforgeeks.org/python/a-search-algorithm-in-python/>  
<https://docs.python.org/3/tutorial/classes.html>  
<https://inst.eecs.berkeley.edu/~cs188/textbook/search/informed.html>

Generative AI: ChatGPT (Free)

Prompt 1 - Help with question 4's comparison table:

“My assignment asks to compare a baseline heuristic with a proposed heuristic using A\* search. both are tested on 2 word pairs using frequency cost, how should a simple table be organized to compare solution costs and nodes expanded? What should a short conclusion look like? “

Prompt 2 - Understanding Python string slicing:

“How does python string slicing work when replacing one letter in a word? IE how does state[:i] + letter + state[i+1:] change dog into cog? Explain each part in beginner terms.”